

Final Note

Data Structure

CSE-2321

Mohammed Nazmul Arefin
11-9-2023

SYLLABUS

Section-B (Final Exam: 50 Marks)

Group-A (20 Marks)

4. Queue – its representation; Insertion & deletion in Queue; Deques; Priority Queues. Recursion [Factorial function, Fibonacci sequence, Ackermann function, Towers of Hanoi]

5. Linked list - Linked list & its representation in memory; Traversing, Searching, Insertion & Deletion operation on Linked list; Header linked lists; two-way lists.

Group-B (30 Marks)

6. Complexity of algorithms, Rate of growth: Big O, Ω and Θ notations; Complexity of Linear Search, Binary search & Bubble sort algorithm. **Sorting** - Insertion sort, selection sort, quick sort, merge sort; **Searching** & data modification; **Hashing**: Hash function, collision resolution

7. Tree- Tree terminology; representation of **binary trees** in memory; Traversing binary tree; **Binary search tree**; Insertion & deletion on binary search tree; **Heap**; Insertion & deletion on heap; Heapsort; B trees; General tree; Balanced binary search tree (AVL tree, red-black tree)

8. Graph – graph terminology; representation of graphs – adjacency matrix, path matrix, adjacency list; Traversing a graph – BFS & DFS

4. QUEUE

Recursion

- 1 Let B be an integer array with N elements. Suppose Y is an integer function defined by 2

$$Y(K) = Y(B, N, K) = \begin{cases} 0 & \text{IF } K = 0 \\ Y(K-1) + B(K) & \text{IF } 0 < K \leq N \\ Y(K-1) & \text{IF } K > N \end{cases}$$

Find Y(5) for each of the following array:

- i. N=8, B: 3, 7, -2, 5, 6, -4, 2, 7 ii. N=3, B: 2, 7, -4

[Au22]

Ans Given, N = 8, B = 3, 7, -2, 5, 6, -4, 2, 7

$$\begin{aligned} Y(5) &= Y(5-1) + B(5) && [K = 5 \text{ and } 5 < 8] \\ &= Y(4-1) + B(4) + B(5) \\ &= Y(3-1) + B(3) + B(4) + B(5) \\ &= Y(2-1) + B(2) + B(3) + B(4) + B(5) \\ &= Y(1-1) + B(1) + B(2) + B(3) + B(4) + B(5) \\ &= 0 + B(1) + B(2) + B(3) + B(4) + B(5) && [K = 0] \\ &= 3 + 7 + (-2) + 5 + 6 \\ &= 19 \end{aligned}$$

Given, N = 3, B = 2, 7, -4

$$\begin{aligned} Y(5) &= Y(5-1) && [K = 5 \text{ and } 5 > 3] \\ &= Y(4-1) && [K = 4 \text{ and } 4 > 3] \\ &= Y(3-1) + B(3) \\ &= Y(2-1) + B(2) + B(3) \\ &= Y(1-1) + B(1) + B(2) + B(3) \\ &= 0 + B(1) + B(2) + B(3) && [K = 0] \\ &= 2 + 7 + (-4) \\ &= 5 \end{aligned}$$

- 2 Let N be an integer and suppose G(N) is recursively defined by 3

$$G(N) = \begin{cases} 3 * N & \text{if } N < 5 \\ 2 * G(N-5) + X & \text{otherwise} \end{cases}$$

i) Find the base criteria of G.

ii) Find G(2), G(8) and G(24).

[Here X is the last digit of your ID. For example, if ID is C191085, X will be 5].

[SP22]

Ans

$$\begin{aligned} G(2) &= 3 * 2 = 6 \\ G(8) &= 2 * G(8-5) + 0 \\ &= 2 * G(3) + 0 \\ &= 2 * 3 * 3 + 0 = 18 \\ G(24) &= 2 * G(24-5) + 0 \\ &= 2 * G(16) + 0 \\ &= 2 * 2 * G(16-5) + 0 \\ &= 2 * 2 * G(11) \\ &= 2 * 2 * 2 * G(11-5) + 0 \\ &= 2 * 2 * 2 * 2 * G(6) \\ &= 2 * 2 * 2 * 2 * 2 * G(6-5) + 0 \\ &= 2 * 2 * 2 * 2 * 2 * G(1) \\ &= 2 * 2 * 2 * 2 * 2 * 3 * 1 \\ &= 96 \end{aligned}$$

- 3 Let A and B be non negative integers. Suppose a function GCD is recursively defined as follows- 2

$$\text{GCD}(A,B) = \begin{cases} \text{GCD}(B,A) & \text{if } A < B \\ A & \text{if } B = 0 \\ \text{GCD}(B, \text{MOD}(A,B)) & \text{Otherwise} \end{cases}$$

Here MOD (A, B) denotes the remainder when A is divided by B.

- i) Find GCD (6,15), GCD (20,28) and GCD (540,168).
ii) What does this function do?

[SP19, SP18]

Ans

$\begin{aligned} \text{GCD}(6, 15) &= \text{GCD}(15, 6) \quad [As, A < B] \\ &= \text{GCD}(6, \text{MOD}(15, 6)) \\ &= \text{GCD}(6, 3) \\ &= \text{GCD}(3, \text{MOD}(6, 3)) \\ &= \text{GCD}(3, 0) \\ &= 3 \end{aligned}$	$\begin{aligned} \text{GCD}(20, 28) &= \text{GCD}(28, 20) \quad [As, A < B] \\ &= \text{GCD}(28, \text{MOD}(28, 20)) \\ &= \text{GCD}(28, 8) \\ &= \text{GCD}(8, \text{MOD}(28, 8)) \\ &= \text{GCD}(8, 4) \\ &= \text{GCD}(4, \text{MOD}(8, 4)) \\ &= \text{GCD}(4, 0) \\ &= 4 \end{aligned}$
---	--

- 4 Let M and N be integers and suppose F(M, N) is recursively defined by 4

$$F(M,N) = \begin{cases} 1 & \text{if } M=0 \text{ or } M \geq N \geq 1 \\ F(M-1, N) + F(M-1, N-1) & \text{Otherwise} \end{cases}$$

Find the value of F(4, 2) and F(2, 4)

[SP18]

Ans

1. $F(4, 2) = 1$
2.
$$\begin{aligned} F(2, 4) &= F(2-1, 4) + F(2-1, 4-1) \\ &= F(1, 4) + F(1, 3) \\ &= F(1-1, 4) + F(1-1, 4-1) + F(1-1, 3) + F(1-1, 3-1) \\ &= F(0, 4) + F(0, 3) + F(0, 3) + F(0, 2) \\ &= 1 + 1 + 1 + 1 = 4 \end{aligned}$$

- 5 Let N be an integer and suppose H(N) is recursively defined by 3

$$H(N) = \begin{cases} 3 * N & \text{if } N < 5 \\ 2 * H(N-5) + 7 & \text{Otherwise} \end{cases}$$

- (i) Find the base criteria of H
(ii) Find the value of H(4) and H(8)

[AU17]

Circular queue:

Insert into the CQueue: if front = 0, then set rear = front = 1 else if rear = n & front != 1, then set rear = 1 else rear = rear + 1	
Is the CQueue full? if (front = 1 & rear = n) OR, (rear + 1) % n = front	
Delete from the CQueue: if front = rear, then front = rear = 0 else if front = n, then front = 1 else front = front + 1	
How many elements in a CQueue? (rear - front + n) % n + 1	

- 1 Suppose a queue is maintained by a circular array QUEUE with N=12 memory cells. 3
Find the number of elements in QUEUE if
- FRONT=5, REAR=10
 - FRONT=12, REAR=3 and
 - FRONT=6, REAR=6

[Au'22]

Ans

1. Number of elements in the queue = (Rear - Front + n) % n + 1

In this case, Front is 5, Rear is 10, and n is 12. Plug these values into the formula:

$$\text{Number of elements in the queue} = (10 - 5 + 12) \% 12 + 1$$

Now, calculate the numerator first:

$$(10 - 5 + 12) = 17$$

Now, calculate the remainder when you divide 17 by 12:

$$17 \% 12 = 5 + 1 = 6$$

So, there are 6 elements in the circular array queue with Front at 5 and Rear at 10.

2. Number of elements in the queue = (Rear - Front + n) % n + 1

Given, Front = 12, Rear = 3, n = 12 (number of memory cells)

Plug in the values:

$$\text{Number of elements in the queue} = (3 - 12 + 12) \% 12 + 1$$

Calculate the numerator first:

$$(3 - 12 + 12) = 3$$

Now, calculate the remainder when you divide 3 by 12:

$$3 \% 12 = 3 + 1 = 4$$

So, there are 4 elements in the circular array queue with Front at 12 and Rear at 3.

- 2 Suppose a queue is maintained by a circular array QUEUE with N=12 memory cells. Find the number of elements in QUEUE if (i) Front=4, Rear=8 and (ii) Front=10, Rear=3 [SP18]

Ans

1. Number of elements in the queue = $(\text{Rear} - \text{Front} + n) \% n + 1$

In this case, Front is 4, Rear is 8, and n is 12. Plug these values into the formula:

$$\text{Number of elements in the queue} = (8 - 4 + 12) \% 12 + 1$$

Now, calculate the numerator first:

$$(8 - 4 + 12) = 16$$

Now, calculate the remainder when you divide 16 by 12:

$$16 \% 12 = 4 + 1 = 5$$

2. Number of elements in the queue = $(\text{Rear} - \text{Front} + n) \% n + 1$

Given, Front = 10, Rear = 3, n = 12 (number of memory cells)

Plug in the values:

$$\text{Number of elements in the queue} = (3 - 10 + 12) \% 12 + 1$$

Calculate the numerator first:

$$(3 - 10 + 12) = 5$$

Now, calculate the remainder when you divide 5 by 12:

$$5 \% 12 = 5 + 1 = 6$$

So, there are 6 elements in the circular array queue with Front at 10 and Rear at 3.

- 3 Consider the following *circular queue* where Queue is allocated 6 memory cells.

Front = 3, Rear = 5, Queue: , , U, C, S,

Illustrate the Queue, including front and rear, as the following sequences take place:

- i) E is added iii) U, V, A are added
ii) 2 items deleted iv) 3 items are deleted [SP22]

Ans:

- i) E is added:

$$\text{front} = 3, \text{rear} = 5 + 1 = 6$$

Queue:

		U	C	S	E
1	2	3	4	5	6
		↑Front			↑Rear

- i) 2 items are deleted

a. $\text{front} = 4, \text{rear} = 6$

Queue:

			C	S	E
1	2	3	4	5	6
			↑Front		↑Rear

b. $\text{front} = 5, \text{rear} = 6$

Queue:

				S	E
1	2	3	4	5	6
				↑Front	↑Rear

- ii) U,V,A are added

a. $\text{front} = 5, \text{rear} = 1$

Queue:

U				S	E
1	2	3	4	5	6
↑Rear				↑Front	

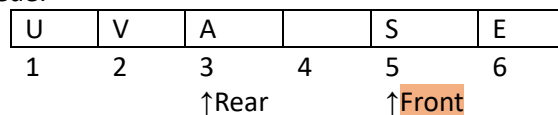
b. $\text{front} = 5, \text{rear} = 2$

Queue:

U	V			S	E
1	2	3	4	5	6
	↑Rear			↑Front	

- c. front = 5, rear = 3

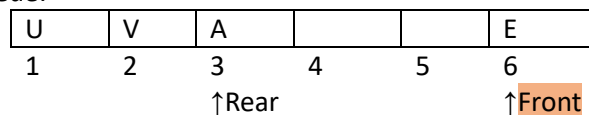
Queue:



- iii) 3 items are deleted

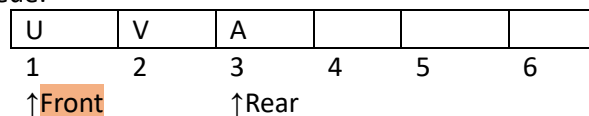
- a. front = 6, rear = 3

Queue:



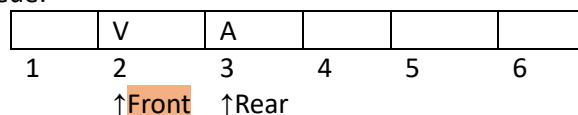
- b. front = 1, rear = 3

Queue:



- c. front = 2, rear = 3

Queue:



4

Consider the following circular queue where Queue is allocated five memory cells:

FRONT=3, REAR=4 QUEUE: , , X, Y,

Illustrate the Queue, including FRONT and REAR, as the following sequences take place:

- i) W is added iii) U, V, A are added
 ii) Two items are deleted iv) One item is deleted

3

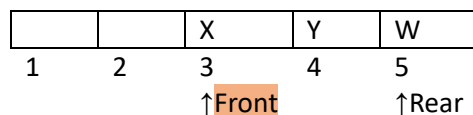
[SP19]

Ans

- i) W is added:

front = 3, rear = 4 + 1 = 5

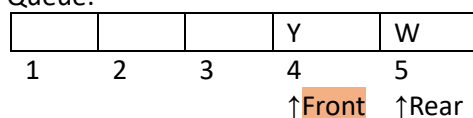
Queue:



- ii) 2 items are deleted

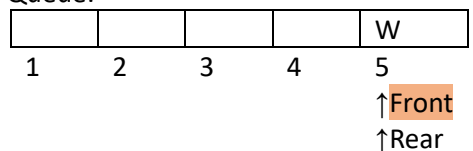
- i) front = 4, rear = 5

Queue:



- ii) front = 5, rear = 5

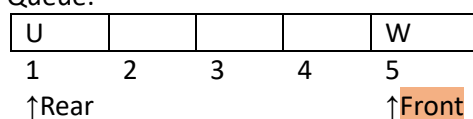
Queue:



- iii) U,V,A are added

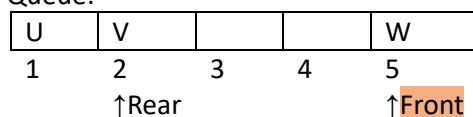
- i) front = 5, rear = 1

Queue:



- ii) front = 5, rear = 2

Queue:



iii) front = 5, rear = 3

Queue:

U	V	A		W
1	2	3	4	5
		↑Rear		↑Front

iv) 1 item is deleted

front = 1, rear = 3

Queue:

U	V	A		
1	2	3	4	5
↑Front		↑Rear		

5 Consider the following queue of characters, where QUEUE is a circular array which is allocated 4 six memory cells:

FRONT=4, REAR=5 QUEUE: _,_,_,D,F_

Describe the queue as the following operations take place:

- (i) K, L and M are added to the queue (ii) Two letters are deleted
(iii) R is added to the queue (iv) One letter is deleted

[AU17]

6 Write a procedure to insert an element into a circular queue and delete an element from a circular queue.

5 [SP22, SP19]

Ans Insertion of an element

1. START
2. Store the element to insert in linear data structure
3. Check if (front == 0 && rear == MAX-1) || (front == rear+1) then queue Overflow else goto step 4
4. Check if (front == -1) then front = 0; rear = 0; else goto step 5
5. Check if (rear == MAX - 1) then rear = 0; else rear = rear + 1; and goto step 6
6. Insert element cqueue_arr[rear] = item;
7. STOP

Deletion of an element

1. START
2. Store the element to delete.
3. If front == -1 then Queue Underflow.
4. Element deleted from queue is cqueue_arr[front].
5. if (front == rear) then front = -1; rear = -1; else goto 6.
6. if (front == MAX - 1) then front = 0 else goto 7.
7. front = front + 1.
8. STOP

7 Why the value of FRONT and REAR are increased during the insertion and deletion element from the queue? (Here queue is represented by linear array).

[AU18]

8 In a circular queue, why does overflow occur when FRONT = 1 and REAR = N, or FRONT = REAR + 1? Explain with example.

What is queue? Describe the representation of circular queue in memory with example.

3 [AU18]

Factorial function, Fibonacci sequence, Ackermann function, Towers of Hanoi

- 1 What is Ackermann function? Use the definition of Ackermann function to find the value of $A(1,3)$. 4

[AU17]

Ans Ackerman function defined as

- If $m = 0$, then $A(m, n) = n + 1$
- If $m \neq 0$ but $n = 0$, then $A(m, n) = A(m-1, 1)$
- If $m \neq 0$ and $n \neq 0$, then $A(m, n) = A(m-1, A(m, n-1))$

$$\begin{aligned}
 A(1, 3) &= A(1-1, A(1, 3-1)) &= A(0, A(0, A(0, 2))) \\
 &= A(0, A(1, 2)) &= A(0, A(0, 2+1)) \\
 &= A(0, A(1-1, A(1, 2-1))) &= A(0, A(0, 3)) \\
 &= A(0, A(0, A(1, 1))) &= A(0, 3+1) \\
 &= A(0, A(0, A(1-1, A(1, 1-1)))) &= A(0, 4) \\
 &= A(0, A(0, A(0, A(1, 0)))) &= 4+1 \\
 &= A(0, A(0, A(0, A(1-1, 1)))) &= 5 \\
 &= A(0, A(0, A(0, A(0, 1)))) \\
 &= A(0, A(0, A(0, 1+1)))
 \end{aligned}$$

- 2 Find the value of $A(2,2)$ using the definition of Ackermann function. 5

[Au'22]

Ans 7

- ◆ Write a recursive procedure for finding *Fibonacci series*. 3 [SP19]
- ◆ Write a recursive procedure for the solution of the *Towers of Hanoi problem*. 3 [SP18]
- ◆ Write an algorithm to find the factorial of a number. 3 [AU17]
- Find the value of factorial five ($5!$) using recursion. Show each step and also show the status of stack that is used in recursion. 5 [Au'22]

Tower of Hanoi consists of three pegs or towers with n disks placed one over the other. The objective of the puzzle is to move the stack to another peg following these simple rules. Only one disk can be moved at a time. No disk can be placed on top of the smaller disk. 5

How many moves does it take to solve the Tower of Hanoi for 5 disks?

◆ [SP22]

- ◆ Graphically show each move of disk in Tower of Hanoi problem for $n=4$. 3 [AU18, AU17]

5. LINKED LIST

◆ Linked list

NOTES DATE: _____

① ways to maintain a list in memory
 → Array
 → Linked list

→ Limitations of array:-

Types of linked list.

① single linked list
 → forward only

② Doubly linked list:
 → forward & backward

③ circular linked list
 → last element is linked to the first element.

④ Single linked list consists of two
 → data
 → link

Diagram illustrating a single linked list node structure:

```

  graph LR
    actual_data[actual data] -- data link --> address[address of the next node of the list]
  
```

♠ Single linked list representation

NOTES DATE: _____

① Representation of single linked list: →

23, 54, 78, 90 to store

Diagram illustrating the representation of a single linked list:

```

  graph LR
    Head[100 Head] --> Node1[23 | 200]
    Node1 --> Node2[54 | 300]
    Node2 --> Node3[78 | 400]
    Node3 --> Node4[90 | Null]
  
```

if we get the first node, we can access all of the data.

to access first node we need a pointer, it is called head.

```

struct node {
    int data;
    struct node * link;
};

int main()
{
    struct node * head = null;
    head = (struct node *) malloc(sizeof(struct node));
    head->data = 45;
    head->link = null;
  
```

Diagram illustrating the memory allocation for the first node:

```

  graph LR
    head[100 head] --> Node[45 | null]
  
```

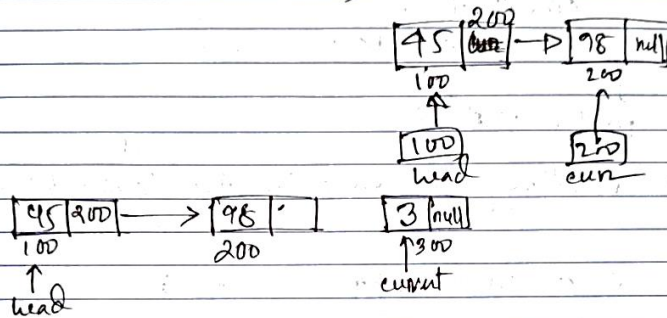
NOTE'S

DATE: _____

```

struct node * cur = malloc (size of (struct
cur -> data = 98;
cur -> link = null;
head -> link = cur;

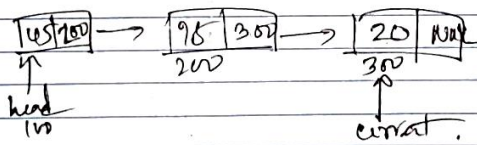
```



```

head -> link -> link = current;

```



Traversing

① Counting nodes in the list -

NOTE'S

DATE: _____

```

int count = 0;
struct node * c = head;

```

```

while (c != null)

```

```

{
    count++;

```

```

    c = c -> link;

```

```

}
print (count);

```

② Printing all data in the ^{single} linked list

```

struct node * p = head;

```

```

while (p != NULL)

```

```

{
    print (p -> data);

```

```

    p = p -> link;

```

```

}

```

♠ Inserting node at the end

★ Insert node at the end: —

NOTES DATE: _____

Diagram showing a linked list with one node containing 5 and NULL. The node is at address 100. A pointer 'ptr' points to the node, and 'Head' also points to it.

Code snippet for inserting a node at the end:

```
ptr = insert_at_end(ptr, 6);
```

```
struct node * insert_at_end(struct node *ptr, int data)
{
    struct node tmp = malloc(sizeof(struct node));
    tmp->data = data;
    tmp->p = NULL;

    ptr->p = tmp; ptr->p->p = tmp;
    return tmp;
}
```

Diagram showing the linked list after insertion. The first node (5, NULL) at address 100 has its 'p' field pointing to a second node (6, NULL) at address 200. The 'ptr' pointer now points to the second node.

Diagram showing the final state of the linked list. The first node (5, 200) at address 100 has its 'p' field pointing to the second node (6, NULL) at address 200. The 'Head' pointer points to the first node, and 'ptr' points to the second node.

Time complexity: $O(1)$

♠ Inserting elements into an array at the end

★ insert element into array at the end

NOTES DATE: _____

Diagram showing an array that is either 'Not full' or 'Full'.

If array is not full, then we can insert an element at the end:

Example: $[1, 3, 5, -, -]$ → $[1, 3, 5, 7, -]$ (this will take $O(1)$ time)

If array is full then we create new empty array with extra space in it & insert older one into it & new element.

Example: $a [1 | 2 | 3]$ → copy to $b [1 | 2 | 3 |]$ → insert new element at the end: $b [1 | 2 | 3 | 6]$

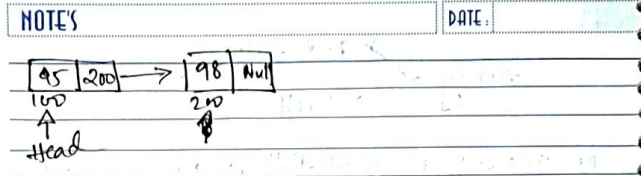
this will take $O(n)$ time complexity

linked	Array
$O(1)$	Full = $O(n)$ Not full = $O(1)$

linked list has this advantage over array

♠ Inserting node at the beginning

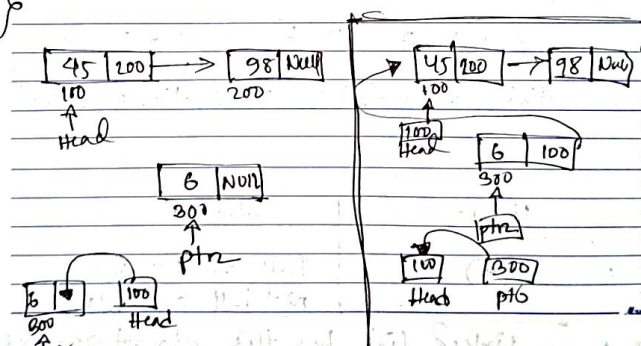
⊛ Inserting the node at the beginning



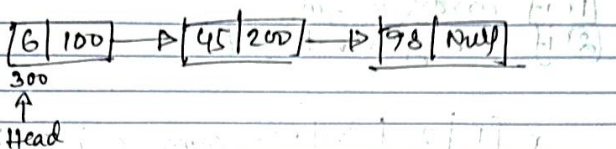
```

struct node * add-at-beg(struct node *head, int data)
{
    struct node *ptr = malloc(sizeof(struct node));
    ptr->data = data;
    ptr->p = NULL;

    ptr->p = head;
    head = ptr;
    return head;
}
  
```



ITE'S _____ DATE: _____



```
main() {
```

```
    struct node * Head = malloc(sizeof(struct node));
```

```
    Head->p = NULL;
```

```
    Head->data = 45;
```

```
    struct node * ptr = malloc(sizeof(struct node));
```

```
    ptr->p = the NULL;
```

```
    ptr->data = 98;
```

```
    Head->p = ptr;
```

```
    Head = add-at-beg(Head, 6);
```

```
    print (Head->p->p->data); // 98.
```

```
}
```

Time complexity $O(1)$

♠ Inserting data at beginning in the array

★ Inserting data at begining in an array.

NOTES

DATE:

(1) if array is full

(2) if array is not full

① →

a [1 | 2 | 3]

Copy a into b
from index 1 of b.
it will take $O(n)$

b [| 1 | 2 | 3]

insert new data here

②

a [1 | 2 | 3 |]

shift all elements
towards right, to
make first place
empty.

a [| 1 | 2 | 3]

insert new data

It will take $O(n)$

linked

Array

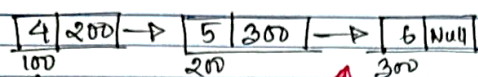
add @ beg: $O(1)$ add @ beg: $O(n)$

♠ Inserting node at any position

★ Inserting data @ any position

NOTES

DATE:



Head

Here we want to add 7

we need to add a value at position 3.

```
void add-at pos(struct node *head, int data, int pos)
{
```

```
    struct node *ptr = malloc(sizeof(struct node));
```

```
    ptr->data = data;
```

```
    ptr->p = Null;
```

```
    struct node *ptr1 = head;
```

```
    int n = 1;
```

```
    while (n != (pos-1))
```

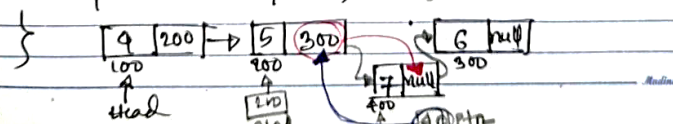
```
    {
        ptr1 = ptr1->p;
```

```
        n++;
```

```
    }
```

```
    ptr->p = ptr1->p;
```

```
    ptr1->p = ptr;
```



- 1 What is the space complexity of a linked list, circular linked list, and doubly linked list for storing N size items of data? 4

[SP19, AU18]

Data Structure	Space Complexity	Explanation
Singly Linked List	$O(n)$	In a singly linked list, each element is stored in a node, which requires $O(n)$ additional memory for the nodes themselves. No additional data structures are needed for the sorting.
Circular Linked List	$O(n)$	Similar to the singly linked list, a circular linked list also requires $O(n)$ space for the nodes. However, it might require a little extra space for a pointer to indicate the start or end, which can be considered as a constant factor and does not affect the overall space complexity.
Doubly Linked List	$O(n)$	A doubly linked list, which has both forward and backward pointers in each node, requires $O(n)$ additional memory for the nodes. Like the singly linked list, no additional data structures are needed for sorting.

- 2 What is *linked list*? What are the advantages and disadvantages of linked list over the linear array. 2

[AU22, SP22, SP18]

Aspect	Linked List	Linear Array
Dynamic Size	- Dynamic size: Linked lists can easily grow or shrink by adding or removing nodes. This is particularly useful when the size of the data structure is unknown in advance. - Time Complexity: $O(1)$ for insertion and deletion at the beginning (for singly linked lists). $O(1)$ for insertion and deletion at both ends (for doubly linked lists).	- Fixed Size: Linear arrays have a fixed size determined during initialization, making it less flexible when the size of the data structure needs to change. - Time Complexity: $O(n)$ for insertion or deletion at arbitrary positions since it may require shifting elements.
Efficient Insertions/Deletions	- Efficient insertions and deletions at any position: Linked lists allow for $O(1)$ insertions and deletions at the beginning, given a reference to the node. - Time Complexity: $O(n)$ for search operations.	- Efficient access and search: Linear arrays allow for $O(1)$ time complexity for accessing elements using an index, making them efficient for read operations. - Time Complexity: $O(1)$ for insertions or deletions at the end, and $O(n)$ for insertions or deletions at arbitrary positions.
Memory Efficiency	- Memory-efficient: Linked lists only allocate memory for the elements they contain, which is advantageous when dealing with a variable number of items. - Time Complexity: $O(n)$ for sequential access.	- Space overhead: Linear arrays may have a fixed, unused space or be too small for the data, leading to wasted memory. - Time Complexity: $O(1)$ for sequential access.

- 3 Let LIST be a *sorted* linked list. Write an algorithm to *search* an element from the LIST. 3

[AU22, SP18]

- 4 Let LIST be a sorted linked list in memory. Write an algorithm to insert an ITEM in the sorted linked list LIST. 4

[AU17]

- 13 The following list of names is assigned (in order) to a linear array INFO: 4
 Mary, June, Barbara, Paula, Diana, Audrey, Karen, Nancy, Ruth, Eileen, Sandra, Helen. That is
 INFO[1]=Mary, INFO[2]=June,....., INFO[12]=Helen. Assign values to an array LINK and a
 variable START so that INFO, LINK and START form an alphabetical listing of the names.

Start



	1	2	3	4	5	6	7	8	9	10	11	12
INFO	Mary	June	Barbar	Paula	Diana	Audrey	Karen	Nancy	Ruth	Eileen	Sandra	Halen
LINK	8	7	5	9	10	3	1	4	11	12	0	2

- 14 Given is the following sorted linked list where the array INFO contains a list of integer numbers and LINK, START and AVAIL are the pointer fields-

	1	2	3	4	5	6	7	8	9	10	11	12
INFO:			41	129		9	123	78	194	231	62	145
LINK:	2	5	11	12	0	3	4	7	10	0	8	9
START : 6 AVAIL : 1												

Determine the changes in the list if number XX is *added* to the list and then 123 is *deleted* from the list.
 [Here XX is the last two digits of your ID]

- 15 Form a two-way list from the one-way list in previous question.

Group B

6. SORT

♣ SORTING

Selection Sort

Selection Sort

DATE: _____

Q1: 0 1 2 3 4
9 13 15 11 2

$A[0]$ is the min compare $A[0]$ with $A[1]$ to $A[4]$
 $A[0]$ is min so swap $A[0]$ with $A[4]$

Sorted: 2 13 15 11 9
 0 1 2 3 4

$A[1]$ compared with others & swap with minimum

Sorted: 2 9 15 11 13
 0 1 2 3 4

Sorted: 2 9 11 15 13
 0 1 2 3 4

Sorted: 2 9 11 13 15
 0 1 2 3 4

```

for (i = 1; i < n; i++)
{
    temp = A[i];
    j = i - 1;

    while (j > 0 && A[j] > temp)
    {
        A[j+1] = A[j];
        j--;
    }

    A[j+1] = temp;
}

```

Time: $j = 0, \text{comp} = 1$
 $j = 1, \text{comp} = 2$
 $j = 3, \text{comparison} = 3$
 $j = n-1, \text{comp} = n$

so, $1+2+3+\dots+n = \frac{n(n+1)}{2} = \frac{n^2+n}{2}$
 $O(n^2)$

Insertion Sort

Insertion Sort

DATE: _____

Sorted: 43 26 29 12
 0 1 2 3 4

temp = 31

compare temp with sorted list from Right to Left

0 1 2 3 4
 31 43 26 29 12

temp = 26

compare temp with $A[1]$ & swap if $A[1] > \text{temp}$
 compare temp with $A[0]$ & swap if $A[0] > \text{temp}$

0 1 2 3 4
 26 31 43 29 12

temp = 29

0 1 2 3 4
 26 29 31 43 12

temp = 12

0 1 2 3 4
 12 26 29 31 43

```

for (int i = 0; i < n-1; i++)
{
    min = i;

    for (j = i+1; j < n; j++)
    {
        if (A[j] < A[min])
            min = j;
    }

    if (min != i)
    {
        temp = A[i];
        A[i] = A[min];
        A[min] = temp;
    }
}

```

Time:-

Iter 1, comp = 4
 Iter 2, comp = 3
 Iter 3, comp = 2
 Iter 4, comp = 1

$1+2+3+4+\dots+n = \frac{n^2+n}{2}$
 $O(n^2)$

Suppose the following numbers are stored in an array A:

22, 33, 29, 49, 21, 57, 62, 73, 54, 44

3

- ◆ Apply *selection sort* algorithm to sort the array A and show each pass separately.

[AU22]

Suppose the following characters are stored in an array A:

B, A, N, G, L, A, D, E, S, H

- ◆ Apply *selection sort* algorithm to sort the array A and show each pass separately.

4 [SP22]

Suppose the following characters are stored in an array A:

D, A, T, A, S, T, R, U, C, T, U, R, E, S

- ◆ Apply *insertion sort* algorithm to sort the array A and show each pass separately.

4 [SP22]

Suppose the following numbers are stored in an array A:

39, 23, 9, 58, 80, 91, 42, 63, 100, 18, 97, 67

- ◆ Apply *insertion sort* algorithm to sort the array A and show each pass separately.

[AU17]

- ◆ State the algorithm to sort an array using *selection sort*. Also analyze complexity.

4 [SP19]

Write an algorithm to sort an array A of n elements using *insertion sort*.

3

[AU22]

Find the total number of exchanges when you implement the *selection sort* algorithm to the following list of numbers.

<77,33,44,11,88,22,66,55>

[AU17, SP18]

Show the operation of the *merge sort* algorithm on the following array

- ◆ A = <44, 33, 11, 55, 77, 90, 40, 60, 99, 22, 88, 66>

[SP18]

Show the operation of the *merge sort* algorithm on the following array

- ◆ A = <44, 33, 11, 55, 77, 90, 40, 60, 99, 22, 88, 66>

[AU17]

♣ HASHING

The following values are to be stored in a *hash table*:

25, 42, 96, 101, 102, 162, 197, 2XX

3

Describe how the values are hashed by using *division method* of hashing with a *table size* of 11. Use **linear probing** as the method of *collision resolution*.

[Here XX is the last two digits of your ID. For example, if ID is C191085, 2XX will be 285].

[AU22]

In the context of hashing, define the terms *bucket*, *slot*, *synonym*, *collision*, *overflow*, *identifier density* and *loading density*.

[SP19]

- ◆ What does a *hash function* do? What are two desirable properties of a *hash function*?

[AU18]

♣ COMPLEXITY

Research the following algorithms to compare their complexity: 4

- i) Linear Search Algorithm
- ii) Bubble Sort Algorithm
- iii) Binary Search Algorithm
- iv) Insertion Sort Algorithm
- v) Selection Sort Algorithm

[SP22]

What do you mean by complexity of an algorithm? How do you calculate the complexity of an algorithm? Describe with example. 3

[AU18]

Analysis the complexity of insertion sort algorithm when the algorithm is implemented in a sorted list of n elements. 3

[AU18,

SP18]

7. TREE

♣ BINARY TREE

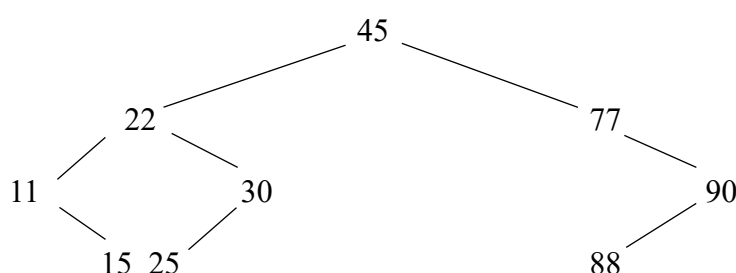
A binary tree T is defined as a finite set of elements called nodes such that,

1. T is empty
2. T contains a distinguished node R , called the root of T and the remaining nodes of T form an ordered pair of disjoint binary trees T_1 and T_2

Sequential representation of binary tree:

Let T be a binary tree that is complete or nearly complete. Then T can be represented using a single linear array $TREE$ such that,

1. The Root R of T is stored in $TREE[1]$
2. If a node N occupies $TREE[K]$, then its left child is stored in $TREE[2K]$ and right child in $TREE[2K+1]$.



1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
45	22	77	11	30		90		15	25					88

Sequential representation of binary tree

Binary tree traversal

1. In order: Left → Root → Right
2. Pre order: Root → Left → Right
3. Post order: Left → Right → Root

Traversal of the tree above:

In order: 11, 15, 22, 25, 30, 45, 77, 88, 90

Pre order: 45, 22, 11, 15, 30, 25, 77, 90, 88

Post order: 15, 11, 25, 30, 22, 88, 90, 77, 45

- 1 Suppose the following sequences list the nodes of a binary tree T in preorder and inorder. 4 respectively;

Preorder: G, B, Q, A, C, K, F, P, D, E, R, H

Inorder: Q, B, K, C, F, A, G, P, E, D, H, R

Draw the diagram of the tree. [AU18]

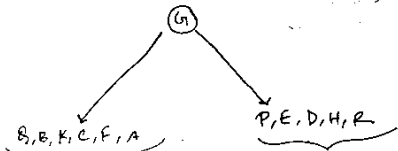
- ♣ We know that,
 For Pre order: Root → Left → Right
 For in order: Left → Root → Right
 So, we can get root for the tree from the pre order.

In the pre order list, first element is G. So, the root of the tree is G

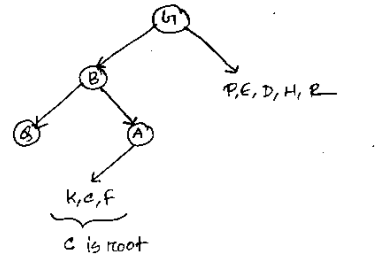
Now, from the in-order list, we can divide the list into two subtrees by G.

			G, B, Q, A, C, K, F, P, D, E, R, H		
Q, B, K, C, F, A			G		P, E, D, H, R
Q	B	K, C, F, A			P, E, D, H, R
Left subtree			Root		Right subtree

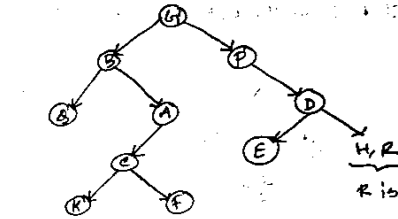
G, B, Q, A, C, K, F, P, D, E, R, H													
Q, B, K, C, F, A						G			P, E, D, H, R				
Q	B	K, C, F, A							P	E, D, H, R			
Q	B	K, C, F			A				P	E	D	H, R	
Q	B	K	C	F	A				P	E	D	H	R
Left subtree						Root			Right subtree				



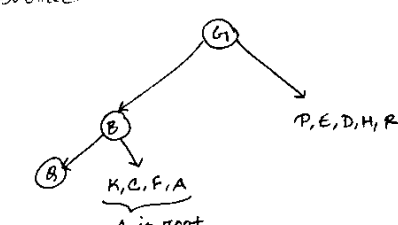
 B is the 2nd element of preorder list. So, E is root here. S is placed in left subtree and (3-6) are placed in right subtree.



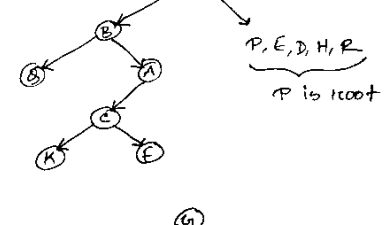
 C is root



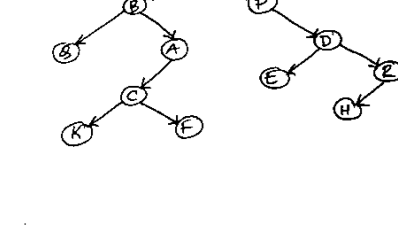
 R is root



 A is root

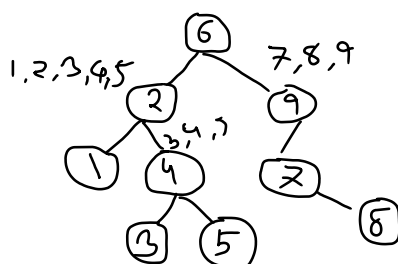


 P is root



 D is root

- [AU17]



- 3 A binary tree T has 9 nodes. The *inorder* and *preorder* traversals of T yields the following sequences of nodes:

Inorder : E A C K F H D B G

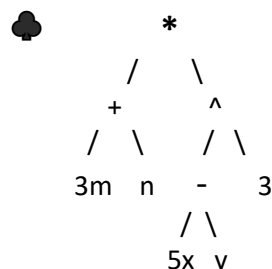
Preorder : F A E K C D H G B

Draw the tree T.

[SP18]

- 4 Consider the algebraic expression $E = (3m + n) (5x - y)^3$.
- a) Draw the tree T which corresponds to the expression E.
- b) Find the *preorder* and *postorder* of T.

3 [AU22, SP19, SP18]



Pre-order: *, +, 3m, n, ^, -, 5x, y, 3

Post-order: 3m, n, +, 5x, y, -, 3, ^, *

- 5 Analyze the differences between Binary tree and Complete binary tree with figure. Construct an *expression tree* T for the following arithmetic expression:

$$(A + B * C) + ((D / E - F) * G)$$

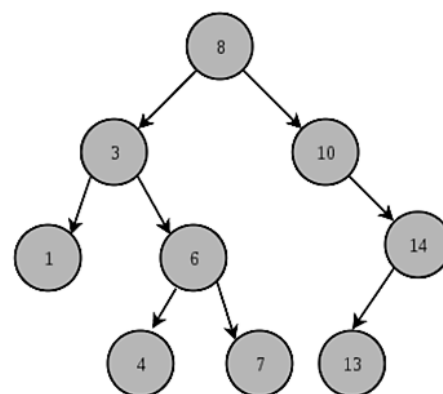
Traverse the tree T in *preorder*, *postorder* and *inorder*.

5 [SP22]

♣ BINARY SEARCH TREE

A binary Search Tree is a binary tree that has the following properties:

- The left subtree of a node contains only nodes with keys lesser than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- The left and right subtree each must also be a binary search tree.
- There must be no duplicate nodes.



Difference between Binary Tree and Binary Search Tree:

#	Basis	Binary Tree	Binary Search Tree
1.	Definition	BINARY TREE is a nonlinear data structure where each node can have at most two child nodes.	BINARY SEARCH TREE is a node based binary tree that further has right and left subtree that too are binary search tree.
3.	Structure	In BINARY TREE there is no ordering in terms of how the nodes are arranged	In BINARY SEARCH TREE the left subtree has elements less than the nodes element and the right subtree has elements greater than the nodes element.
4.	Data Representation	Data Representation is carried out in a hierarchical format.	Data Representation is carried out in the ordered format.

#	Basis	Binary Tree	Binary Search Tree
5.	Duplicate Values	Binary trees allow duplicate values.	Binary Search Tree does not allow duplicate values.
7.	Complexity	$O(n)$.	$O(\log n)$.
8.	Application	It is used for retrieval of fast and quick information and data lookup.	It works well at element deletion, insertion, and searching.
9.	Usage	It serves as the foundation for implementing Full Binary Tree, BSTs, Perfect Binary Tree, and others.	It is utilized in the implementation of Balanced Binary Search Trees such as AVL Trees, Red Black Trees, and so on.

Insertion of elements into Binary Search Tree:

During insertion, we must consider following conditions

- Left < Root
- Right > Root

1 What is a *binary search tree*? Suppose the following eight numbers are inserted in order into an empty binary search tree.

40 23 34 22 77 45 60 50

i) Draw the tree T, ii) Find the *inorder* traversal of T.

[SP'18]

OTES Insert into BST

40, 23, 34, 22, 77, 45, 60, 50

1. Insert 40 as root

2. Insert 23: 23 < 40

3. Insert 34: 34 < 40, 34 > 23

4. Insert 22: 22 < 40, 22 < 23

5. Insert 77: 77 > 40

6. Insert 45: 45 > 40, 45 < 77

7. Insert 60: 60 > 40, 60 < 77, 60 > 45

8. Insert 50: 50 > 40, 50 < 77, 50 > 45, 50 < 60

In order traversal: 22, 23, 34, 40, 45, 50, 60, 77

2 Suppose the following nine numbers are inserted in order into an empty binary search tree T:

50, 33, 44, 22, 77, 35, N, 60, 40,

i. Draw the tree T.

ii. Traverse the tree T in *preorder*, *postorder* and *inorder*.

(Note: N is the last two digits of your ID. i.e. if your ID is C151216, then N=16)

[AU22]

3 What is a *binary search tree*? Suppose the following eight numbers are inserted in order into an empty binary search tree.

49, 42, 19, 62, 37, 79, 44, 8

i) Draw the tree T.

ii) Find the *inorder* traversal of T.

4 [SP19]

4 What is a *binary search tree*? Suppose the following eight numbers are inserted in order into an empty binary search tree.

77 98 2 33 4 56 88 9

Draw the tree T

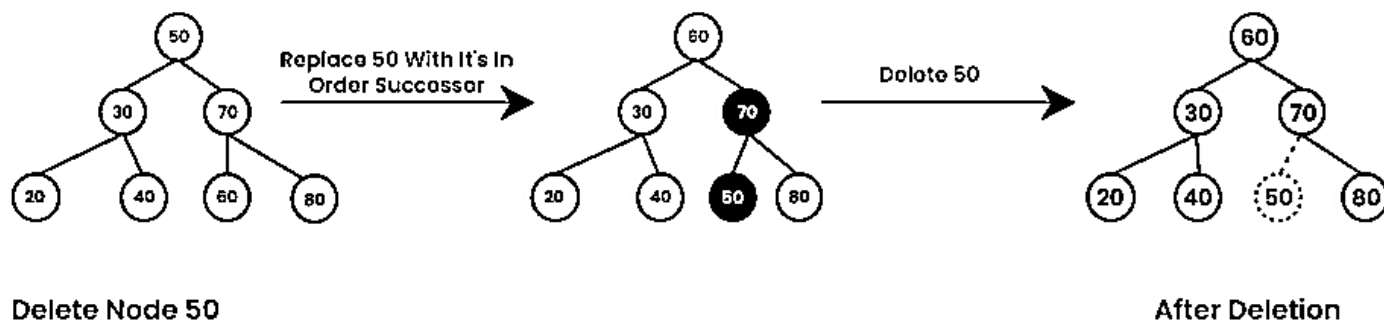
[AU17]

- 5 Suppose the following list of letters is inserted in order into an empty binary search tree:
J, R, D, G, T, E, M, H, P, A, F, Q
(i) Find the final tree T, (ii) Find the inorder traversal of T. [SP18]
- 6 What is a *binary search tree*? Suppose the following eight numbers are inserted in order into an empty binary search tree.
49, 42, 19, 62, 37, 79, 44, 8
i) Draw the tree T.
ii) Find the inorder traversal of T. 4 [SP22]
- 7 When we apply the inorder traversing in a binary search tree, the list of visiting nodes is a sorted list. Do you agree with this statement? Justify your answer. (Note that each node of this binary search tree contains a positive integer) [AU18]

Deletion of elements from Binary search tree:

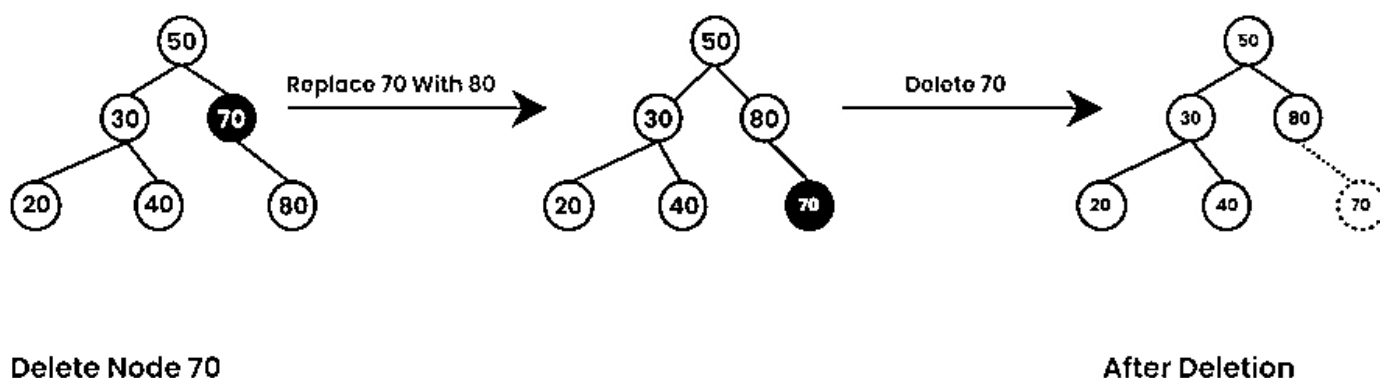
During deletion we can follow any two ways if the node has two children (left and right).

- Predecessor: We can find the **largest** value from the **left sub-tree** and replace the value with the node
 - Successor: We can find the **smallest** value from the **right sub-tree** and replace the value with the node
- If there is only one child, we can simply replace the value with the node we want to delete and remove the node.



In order successor is needed only when the right child is not empty. In this particular case, the in-order successor can be obtained by finding the minimum value in the right child of the node.

Delete node with single child:

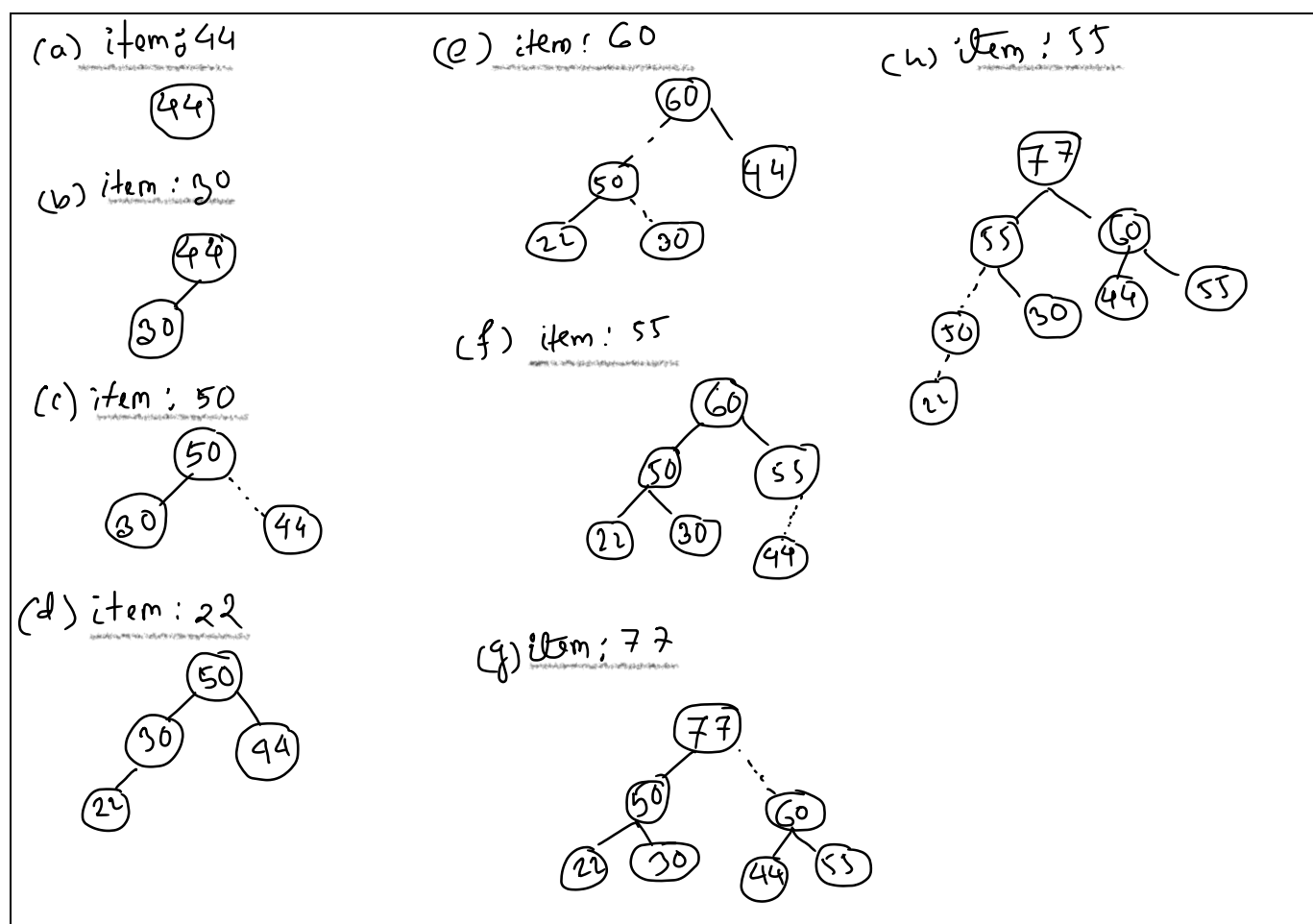


♣ HEAP

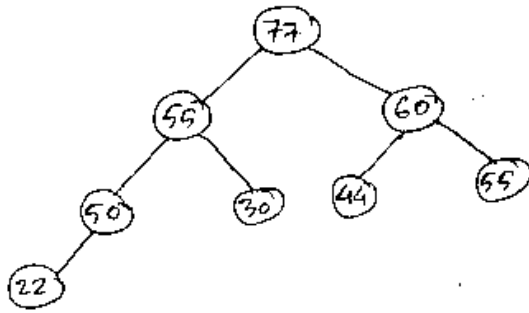
Max Heap	Min Heap
For every node i , the value of node is less than or equal to its parent value. $A[\text{parent}(i)] \geq A[i]$ → Root > children's values → Must be complete binary tree	For every node i , the value of node is greater than or equal to its parent value. $A[\text{parent}(i)] \leq A[i]$ → Root < children's values → Must be complete binary tree
<pre> graph TD 70 --> 50 70 --> 40 50 --> 45 50 --> 35 45 --> 10 45 --> 9 40 --> 39 40 --> 16 </pre>	<pre> graph TD 10 --> 15 10 --> 19 15 --> 20 15 --> 30 20 --> 23 20 --> 40 30 --> 35 19 --> 25 19 --> 39 </pre>

Insertion in Max heap:

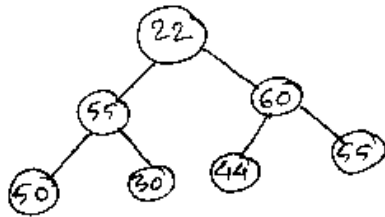
Build a max heap from the following list of numbers: 44, 30, 50, 22, 60, 55, 77, 55

**Deletion from max heap:**

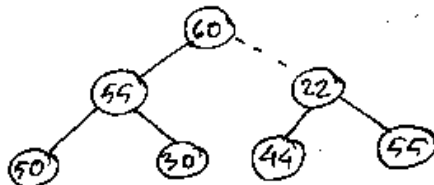
Delete 77 from the max heap.



Moving 22 to root,



Sink 22 to the right place,



- What do you mean by *max heap*? Build a max heap from the following list of numbers: 3
 ♦ 36, 39, 29, 27, 37, 52, 63, 44 [AU22]

- What do you mean by *max heap*? Build a max heap from the following list of numbers (Show each step):
 ♦ 33, 29, 49, 21, 57, 62, 73, 54 4 [SP22]

- What do you mean by *min heap*? Build a min heap from the following list of numbers:
 ♦ 39, 35, 10, 60, 99, 22, 82, 69 [AU17]

- ♦ What are Heap, Maxheap and Minheap? Give the benefits of Heap Structure. Describe about Heapsort. [AU18]

8. GRAPH

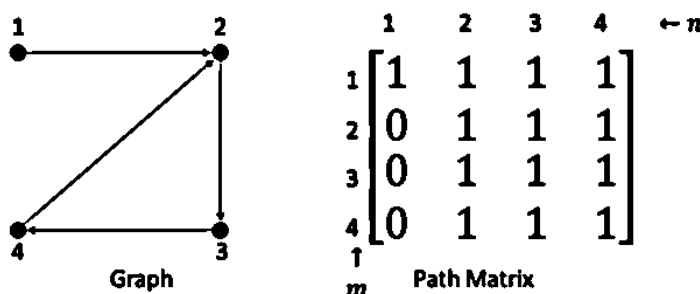
1 **Define:** Graph, Adjacency matrix, adjacency list, path matrix

- ♣ **Graph:** A graph consist of two things:
1. A set N of elements called nodes or vertices
 2. A set E of edges such that each edge e in E is identified with a unique (unordered) pair $[u, v]$ of nodes in V , denoted by $e = [u, v]$

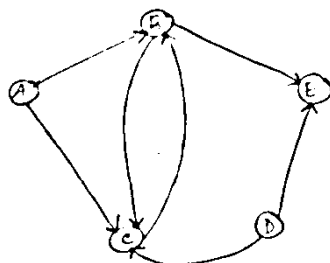
Adjacency matrix: In graph theory, an adjacency matrix is nothing but a square matrix utilized to describe a finite graph.

Adjacency List: An adjacency list is a data structure used to represent a graph where each node in the graph stores a list of its neighboring vertices.

Path matrix: A path matrix is a matrix representing a graph, where each value in m^{th} row and n^{th} column project whether there is a path from node m to node n . The path may be direct or indirect. It may have a single edge or multiple edge.



2 **Give adjacency matrix, adjacency list of the following graph**



♣ **Adjacency matrix:**

	A	B	C	D	E
A	0	1	1	0	0
B	0	0	1	0	1
C	0	1	0	0	0
D	0	0	1	0	1
E	0	0	0	0	0

Adjacency List:

Nodes : Neighbours

A : B, C
 B : C, E
 C : B
 D : C, E
 E : (empty)

3 Define 1) Simple graph 2) Complete graph

**Simple graph**

A directed graph is said to be simple if G has no parallel edges. A simple graph G may have loops but it can't have more than one loop at a given node.

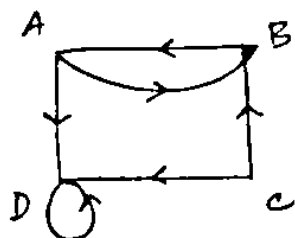


Figure: Simple graph

Complete graph

A graph G is said to be complete if every node u in G is adjacent to every other node v in G .

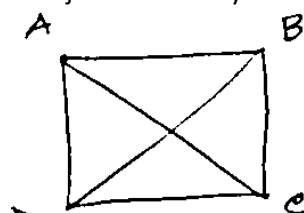


Figure: Complete graph

4 Define graph? Draw a picture of the directed graph specified below:

$G = (V, E)$

$V(G) = \{1, 2, 3, 4, 5, 6\}$

$E(G) = \{(1,2), (2,3), (3,4), (5,1), (5,6), (2,6), (1,6), (4,6), (2,4)\}$

Obtain the following for the above graph:

- Find the *adjacency matrix* A of the graph G .
- Find the *adjacency list* of the graph G .

3 [sp22]

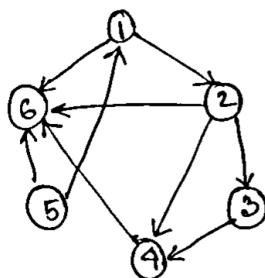


Figure: Graph G

i) *Adjacency Matrix*:

	1	2	3	4	5	6
1	0	1	0	0	0	1
2	0	0	1	1	0	1
3	0	0	0	1	0	0
4	0	0	0	0	0	1
5	1	0	0	0	0	1
6	0	0	0	0	0	0

ii) *Adjacency List*:

1: 2, 6
 2: 3, 4, 6
 3: 4
 4: 6
 5: 1, 6
 6: 1

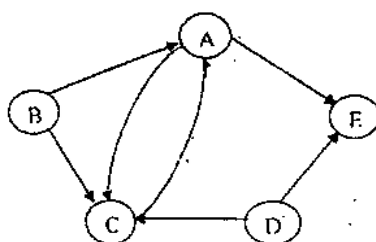
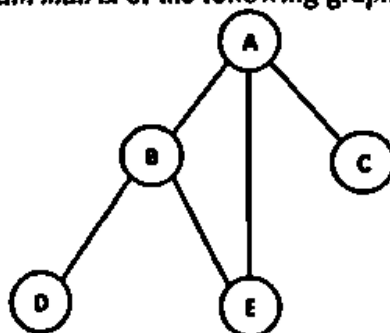
5 What is a graph? Consider the following graph G in figure 1:

Figure 1

- Find the *adjacency matrix* A of the graph G .
- Find the *adjacency list* of the graph G .

3 [SP19, SP18]

6 Find the *adjacency matrix* and *path matrix* of the following graph

[AU17]

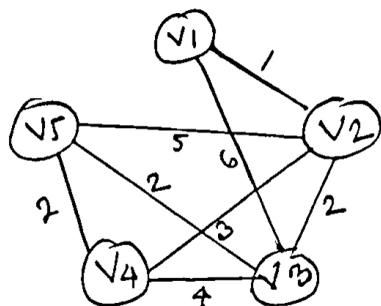
- 7 Suppose a weighted graph G is maintained in memory by a node array $DATA$ and a weight matrix W as follows:

$DATA: V_1, V_2, V_3, V_4, V_5$

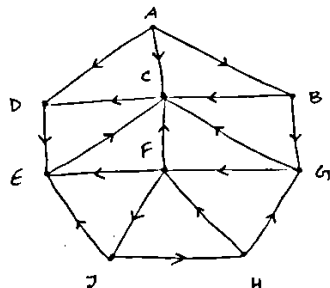
	0	1	6	0	0
	1	0	2	3	5
$W =$	6	2	0	4	2
	0	3	4	0	2
	0	5	2	2	0

Draw the graph G .

[AU17]



- 8 BFS and DFS for the following graph consider A as the source:



- The general idea behind Breadth First Search (BFS) beginning at a starting node A is as follows. First we examine the starting node A. Then we examine all the neighbors of A. Then we examine all the neighbors of the neighbors of A and so on.

Let us assume we want to traverse the graph using BFS starting at A. We will use QUEUE to keep track of neighbor nodes.

First push A to QUEUE and mark A as visited and pop A, Then Push neighbors of A to QUEUE.

QUEUE: B, C, D

Pop B, Push neighbors of B and mark B as visited.

QUEUE: C, D, E

Pop C, Push neighbors of C and mark C as visited.

QUEUE: D, G

Pop D, Push neighbors of D and mark D as visited.

QUEUE: G, E

Pop G, Push neighbors of G and mark G as visited.

QUEUE: E, F

Pop E, Push neighbors of E and mark E as visited.

QUEUE: F

Pop F, Push neighbors of F and mark F as visited.

QUEUE: J

Pop J, Push neighbors of J and mark J as visited.

QUEUE: H

Pop H, Push neighbors of H and mark H as visited.

QUEUE: (empty)

As we can see the queue is empty, that means we've visited all nodes in the graph.

We start from vertex A, the DFS algorithm starts by putting it in the Visited list and putting all its adjacent vertices in the stack. So, put A in the visited list and B, C, D in the stack.

Adjacency List	Nodes	Marker
A	B, C, D	A
B	C, G	B
C	D	C
D	E	D
E	C	E
F	C, E, J	F
G	C, F	G
H	F, G	H
J	E, H	J

A									Visited
B	C	D							Stack

Now, in the top of the stack we have B

So, Set B as visited and put G in the stack. [As C is already in the stack, we won't put it again]

A	B								Visited
G	C	D							Stack

Now, in the top of the stack we have G

So, Set G as visited and put F in the stack. [As C is already in the stack, we won't put it again]

A	B	G							Visited
F	C	D							Stack

Now, in the top of the stack we have F

So, Set F as visited and put E, J in the stack. [As C is already in the stack, we won't put it again]

A	B	G	F						Visited
E	J	C	D						Stack

Now, in the top of the stack we have E

So, Set E as visited. [As C is already in the stack, we won't put it again]

A	B	G	F	E					Visited
J	C	D							Stack

Now, in the top of the stack we have J

So, Set J as visited and put H in the stack.

A	B	G	F	E	J				Visited
H	C	D							Stack

Now, in the top of the stack we have H

So, Set H as visited.

A	B	G	F	E	J	H			Visited
C	D								Stack

Now, in the top of the stack we have C

So, Set C as visited.

A	B	G	F	E	J	H	C		Visited
D									Stack

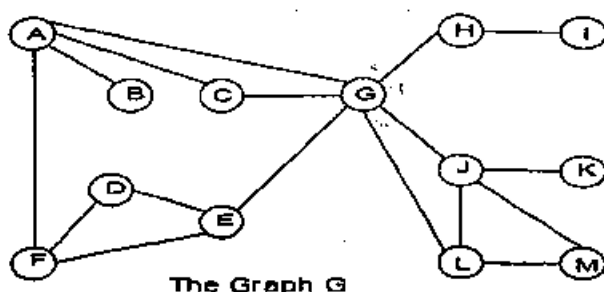
Now, in the top of the stack we have D

So, Set D as visited.

A	B	G	F	E	J	H	C	D	Visited
									Stack

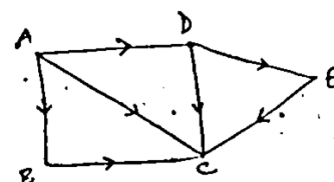
After we visit the last element, stack doesn't have any unvisited adjacent nodes, so we have completed the Depth First Traversal of the graph.

- 9 **Traverse the graph G shown below in breadth first order, depth first order and construct the breadth first and depth first spanning trees. Start from node C [if last digit of your ID is odd] / J [if last digit of your ID is even].**



4 [sp22]

- 10 Consider the following graph below, give the linked list representation of the graph.



Adjacency list of this graph:

A: B, C, D

B: C

C:

D: C, E

E: C

The linked list representation for this graph will contain two lists, Node list and Edge list

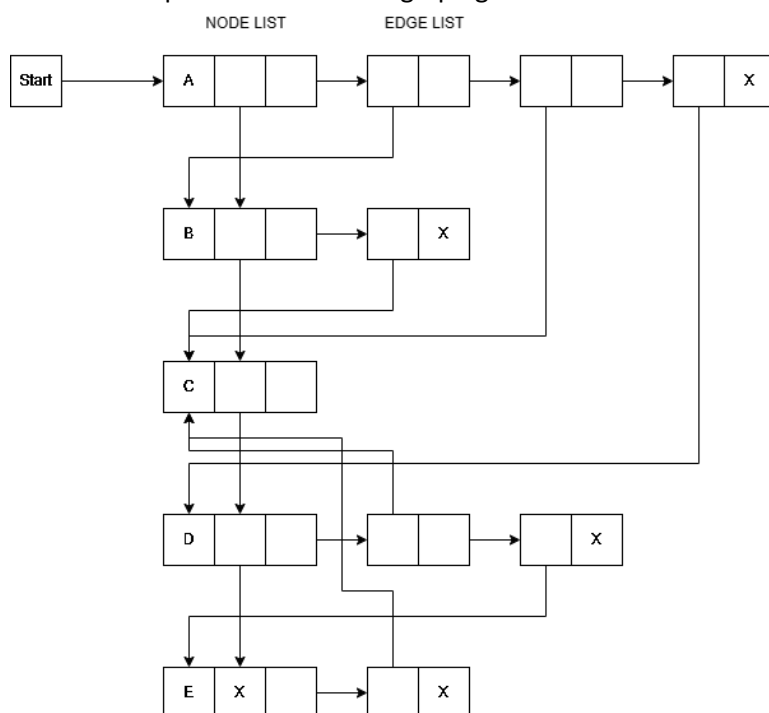
Each element in the Node list will be a record of the form:

NODE	NEXT	ADJ	
------	------	-----	--

Each element in the edge list will correspond to an edge of graph and will be a record of the form:

DEST	LINK	
------	------	--

The linked representation of the graph given below:



11 Give the algorithms for DFS and for BFS of a graph.

12 For a set of vertexes V with n elements, how many possible edges there?
Give a comparison between *breadth first search* and *depth first search*.

The maximum number of edges possible in a single graph with ' n ' vertices is nC_2 where, ${}^nC_2 = n(n-1)/2$

BFS	DFS
BFS starts traversal from root node and then explores the search in the level by level manner	DFS starts traversal from the root node and explore the search as far as possible from the root node i.e. depth wise
BFS can be done using queue.	DFS can be done using stack
It uses FIFO implementation	It uses LIFO implementation
BFS is slower than DFS	DFS is faster

13 Define following terms with necessary figure.

(i) Digraph (ii) Adjacent vertices (iii) Self loop (iv) Complete graph (v) Connected graph (vi) Isolated vertex

14 Define the following term with figure.

(i) Simple graph (ii) Complex graph (iii) Isolated vertex (iv) Complete graph.

15 What is *path matrix*? Consider the graph G in Fig-1. Find the path matrix P of G using *Warshall algorithm*. Compute the matrices P_0, P_1, P_2, P_3, P_4 and P_5 .